

Dart and null safety

Classes, null-safe types and collections

Lecturer

Dinu-Ștefan Rusu

Course

ADMD

Lab objectives

Null safety in code

When to use `?`, `?.`, `??` and `late`

Time
2 hours

Classes and inheritance

Model a small domain with an abstract base class and two subclasses

Present
To the lab instructor, at the end of this lab

File layout

1. Open the project from Lab 1.
2. Create `lib/models/` directory. All classes live there.
3. One class per file with `lower_snake_case` filenames: `user.dart`, `bank_account.dart`, etc.
4. Only `main.dart` holds widget code: just enough to display what your classes produce.

Hint

Dart convention: lowercase filename for lowercase class. You are asked to open these files when you present.

Null-safety types and operators

Types and operators

- `String?`: may be null; plain `String` never can
- `?.`: read a member only if non-null
- `??`: use right-hand value if left is null
- `??=`: assign only if currently null
- `!`: assert non-null; throws if wrong

Declarations and promotion

- `late`: assigned after construction
- `required`: caller must pass it
- `= 0.0`: default for optional parameter
- `is Book`: type test that promotes variable
- `x != null`: flow analysis promotes to non-null

Prefer `?.`, `??` and null checks over `!`. The compiler cannot check a value asserted with `!`.

Null safety example

```
class Room {  
    final String code;           // always present  
    final int? seats;            // may be unknown  
    String? projector;           // may be absent  
    late final DateTime opened;  // set later, read after  
  
    Room({required this.code, this.seats, this.projector});  
  
    String describeSeats() =>  
        seats == null ? 'Unknown' : '$seats seats';  
}
```

`code` never null. `seats`, `projector` may be null, so compiler forces handling. `opened` has no value yet; reading it throws `LateInitializationError`.

Users

1. Create `User` with `String name`, `int? age`, `String? email`.
2. Make `name` required; leave `age`, `email` optional.
3. Create three `User` instances with different null combinations.
4. Display safely; show “Not provided” where value is null.
5. Implement `getDisplayAge()`: return “Age not specified” or age as string.

Done when

- ☐ Three users render, one with all optionals null
 - ☐ Both branches of `getDisplayAge()` work
-

Bank accounts

1. Create `BankAccount`: late `accountNumber`, required `holderName`, `balance = 0.0`.
2. Add optional `bankBranch`.
3. Implement `initializeAccount(String number)`.
4. Add method to check if account number is set.
5. Display accounts safely in `ListView`.

Done when

- ☐ All accounts render with branch or "N/A"
 - ☐ Uninitialized account shows message
-

Minimal UI for the models

```
ListView.builder(  
  itemCount: accounts.length,  
  itemBuilder: (context, i) {  
    final a = accounts[i];  
    return ListTile(  
      title: Text(a.holderName),  
      subtitle: Text(a.bankBranch ?? 'Not provided'),  
    );  
  },  
)
```

Widgets are Lecture 3 and Lab 3. Build visible rows only. `??` handles “Not provided” in widgets.

Library: model classes

1. Abstract `LibraryItem`: `title`, `author?`, `publishDate?`.
2. `Book` extends `LibraryItem`: add `pageCount?`, `isbn?`.
3. `Magazine` extends `LibraryItem`: add `issueNumber?`, `publisher?`.
4. Pass shared fields via super constructor.
5. Add `borrower?` (reuse Task I `User`) and `dueDate?` to `LibraryItem`.

Done when

- ☐ Both subclasses compile without unset fields
 - ☐ All optional fields use `?`
-

Library: API and borrowing

1. `Library` holds `List<LibraryItem>`; add `addItem()` with validation.
2. Implement `searchByTitle()`, `getItemsByAuthor()`, `getRecentItems()` with null handling.
3. Add `borrowItem()` that checks if already borrowed.
4. Add `returnItem()` that clears borrower and `dueDate`.

Done when

- ☐ All methods handle null safely
 - ☐ Borrowed items are protected
 - ☐ Null queries return empty list
-

Common errors

Non-nullable 'x' not assigned before use

Give a value, use `?`, or mark `late`

`LateInitializationError` on 'x'

A `late` field read before it was set

Can't access property on nullable receiver

Use `?.`, `??`, or null check first

Null check `!` used on null value

Replace with `??` or real null check

Type 'String?' can't assign to 'String'

Promote with `if (x != null)` or `??`

Presentation

The app running · your classes, explained